Lecture 1 – AERM Macro

Florian Trouvain

University of Oxford

October 15, 2025

Introduction

- Goal of this class is to expose you to research methods in macro/structural labor.
- Hard to do in 3 weeks — macro nowadays combines methods from many fields.
- I will teach three things:
  1. Julia
  2. Dynamic Programming
  3. Consumption–saving problem with income risk in general equilibrium
  4. How to simulate a model — and match it to micro moments
- Benefit: the basic problem carries over to most other settings.

Problem Sets & Exam

- Weekly problem sets:
  - Mix of theory, coding, and a little bit of empirical work.
  - Submit a PDF (LaTeX/Lyx) + code.
    - Work together if you like but submit independent work.
    - Use AI as much as you want, but make sure you understand what you are doing.
  - I will post the problem set on Friday evening and give you a week to submit.
  - Everything on Canvas.
- Final exam:
  - Take-home.
  - I will post a mock exam in due time, similar to problem sets in weeks 2 and 3.
  - Important: submit independent work — you cannot work together during exam.

Communication

- If you have questions about the material, please ask.
  - The main point of meeting in person is so you can ask questions.
    - If you have organizational questions that matter for everyone, ask in class.
    - Explaining complicated concepts via email is not effective.
  - Code troubleshooting via email is even less effective.
- I will hold office hours on Friday from 16:30–18:00 in office 254, Manor Rd.
  - Office hours are great when you come prepared.

How to Get the Most Out of This Class

- Fundamental understanding beats memorizing.
- You have to do the problem sets to learn how to code.
- It can be a bit painstaking — code that doesn't work can be frustrating.
- Learning vs. your own research:
  - In this class, we write code from scratch.
  - In your own work, I recommend building on other researchers' code.
    - Preferably from someone with some standing in the profession.

Introduction to Julia

Why Julia?

- Fast, simple, free.
- Follow online lecture notes by Sargent, Stachurski, and Perla:

  "Quantitative Economics with Julia" (search on Google).
- They cover everything — perfect to learn more.
  - McCall model — labor search
  - Schelling's model of segregation
  - ...

Julia Basics

- Install Julia and VS Code, and use them together.
  - Like Stata or R: Julia runs in the terminal, and VS Code is where you write your scripts.
- Good coding routine:
  - Split code into many parts and add clean comments.
    1. `run_cake_eating.jl`
    2. `define_params_cake_eating.jl`
    3. `functions_cake_eating.jl`

Terminal in Julia with VS Code

load packages

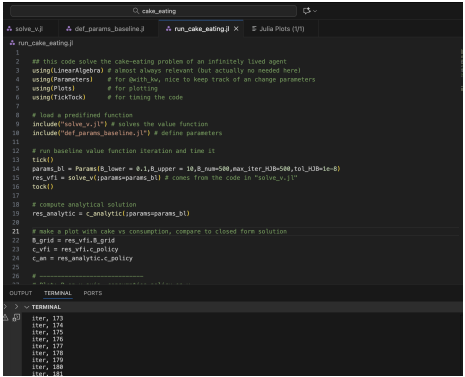

Figure 1: Screenshot: Terminal

- Install and load packages using `using`.
- Include pieces of code — functions you define and then use.
- Highlight a piece of code and use Shift+Return to execute that line.

Common Data Types

- Common types:
  - Integers: 1, 2, 3 Floats: 1.1, 2.0, –3.333333
  - Strings: `"Florian"`
  - Booleans: `true` vs. `false` (1 vs. 0)
- Arrays of type T and dimension N: `Array{T,N}` — general structure for quantitative work.
  - `aux = Array{String}(undef, 2, 2, 2, 2, 2)`
    - `aux[1,1,1,1,2]=4` → No
    - `aux[1,1,1,1,2]="Maria"` → Yes
  - `aux = Array{Float64}(undef, 2, 2); aux[1,:].=1; aux[2,:]=.2`

    → 2×2 `Matrix{Float64}`: [1.0 1.0; 2.0 2.0]

Linear Algebra

- Let $x = [1\ 1; 1\ 1]$, $y = [1\ 2; 3\ 4]$ be two matrices. Then $x * y$ is standard matrix multiplication, $x + y$ is addition.
- Transpose: X' or `transpose(X)`.
- Inverse: x^{-1} or `inv(x)`.
- Make a matrix with ones: `B = ones(2, 2)`.
- Element-wise operation: `B.*B = [1 1; 1 1]`.
- The identity matrix I scales automatically.
- Suppose you want to solve $Xp = b$ (e.g., stationary distn' in continuous time):

  p: stationary distribution

  X: transitional matrix

  - You can compute the inverse of X, but better: `p = X \b`. divide: \ backward slash
  - This solution is much faster than computing the actual inverse.

Functions

- Functions in Julia are amazing.
  - They make your code cleaner and less error-prone.
  - Julia optimizes speed within functions.
  - Any value can be passed to a function as an argument, including other functions.
  - Functions can be (and often are) defined inside other functions.
- Quick function with broadcasting:

$$f(x) = x^2$$

$$y = [1;\ 2;\ 3]$$

$$f.(y) = [1;\ 4;\ 9]$$

Functions (Example)

- Write a function where a lot of stuff happens:
  - Pack parameters into `params` and unpack inside the function.
  - Return a whole array of outputs (stationary distribution, value function, . . .).
  - You can write helper functions inside the main function.

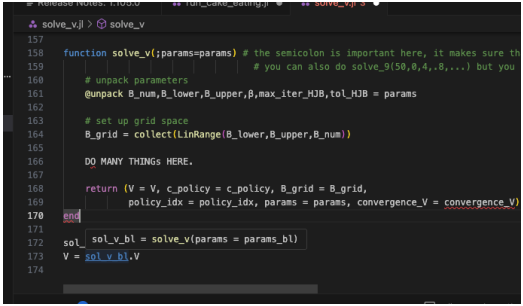

Figure 2: Functions

Some More Julia

- Mutable and immutable objects — be careful:
  - `x=1; y=x; x=2` → what is `y`?
  - `x=1; y=copy(x); x=2` → now what is `y`?
  - `x=[1 ; 1]; y=x; x[1,1]=0` → now what is `y`?
  - `x=[1 ; 1]; y=copy(x); x[1,1]=0` → now what is `y`?
- Changing dimensions:
  - `a = [10, 20, 30, 40]`
  - `b = reshape(a,2,2) = [10 30; 20 40]`
- Tuples are high-performance immutable sets of distinct types:
  - `t = (alpha=1, beta="Oxford")` and `t.alpha = 1`.

Good Practices for Functions and Variable Types

- To keep the benefits of Julia, ensure the compiler can deduce types in any function.
  - Bad: an array holding unrelated types like `x = [1.0, "test", 1]`.
  - The type of this array is `Array{Any,1}` — meaning the compiler assumes any valid Julia type can be added.

Introduction to Dynamic Programming

Gentle Introduction to Dynamic Programming

- Dynamic programming allows you to solve:
  - Dynamic migration choices
  - Models of marriage
  - Investment in schooling
- You can abstract away from this in static problems, but it's necessary for a world that is:
  - Changing over time
  - Stochastic (even if aggregates are stationary)
  - Or both
- These are the contexts in which most empirical estimates are obtained.

Bellman's Principle of Optimality

- Dynamic programming achieves its tractability by shifting attention from histories/sequences to states.
- **Bellman's Principle of Optimality:** An optimal policy has the property that, whatever the initial state and decision (i.e., control) are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision. (Bellman, 1957)
- Try to learn what this means by using dynamic programming.

Cake Eating Problem

- Simple example: the cake-eating problem
- Infinitely lived agent, discounting the future at rate $\beta < 1$, log utility

$$V_0 = \sum_{t=0}^{\infty} \beta^t \log(c_t)$$

- Budget constraint:
  - $b_{t+1} = b_t - c_t, \qquad b_0 > 0$
  - $b_t \geq 0 \quad \forall t$
- Idea: you start out with a cake of size b_0. How fast should you eat it?

Sequence Problem (Calculus Approach)

- **Lagrangian:**

$$\mathcal{L} = \sum_{t=0}^{\infty} \beta^t u(c_t) + \sum_{t=0}^{\infty} \lambda_t \big(b_t - c_t - b_{t+1}\big), \tag{1}$$

  with b_0 given.

- First-order conditions:

$$\frac{\partial \mathcal{L}}{\partial c_t} = 0 \quad \Rightarrow \quad u'(c_t) - \lambda_t = 0, \tag{2}$$

$$\frac{\partial \mathcal{L}}{\partial b_{t+1}} = 0 \quad \Rightarrow \quad -\lambda_t + \beta \lambda_{t+1} = 0. \tag{3}$$

Euler Equation

- Standard Euler equation:

$$\frac{c_{t+1}}{c_t} = \beta$$

Each period, consumption shrinks by the factor β

If β = 0.9, you eat 10% less each period — you're patient, saving some cake for the future.

- Plug into the intertemporal budget constraint — iterate forward and note that

$$b_0 = \sum_t c_t \tag{4}$$

- Pins down optimal c_t: the policy function depends on model primitives:

$$c_t = (1 - \beta)\beta^t b_0 \tag{5}$$

Dynamic Programming

- Basic idea: split the problem into today vs. the future and focus on state.

$$\underbrace{V(b)}_{\text{value function}} = \max_{0 \leq c \leq b} \left\{ \underbrace{\log c}_{\text{flow payoff}} + \underbrace{\beta V(b-c)}_{\text{continuation value}} \right\}. \tag{6}$$

- V is defined as the utility associated with the optimal policy.
  - V is a function.
  - If you knew V, (6) would be an easy "static" two-period problem.
  - Plug this into the sequence problem — the two are identical under regularity conditions.
    - Note: no V_t — this is why macroeconomists like infinite horizons.
    - The future looks like today, conditional on the state b.

Dynamic Programming

- **Magical:** even though V is a complicated, possibly non-linear function, a simple algorithm pins down $V(b)$.
- Why do we care? Most interesting problems cannot be solved with calculus.
  - Borrowing constraints, precautionary savings, ...

Dynamic Programming: Cookbook Now, Theory Later

- Denote b' the amount of cake left for next period — a choice.

$$c = b - b' \tag{7}$$

- Given the budget constraint, it doesn't matter if you pick c or b' — same thing!
- b is predetermined — a state variable.
- Language: control vs. state
  - Control is a jump variable; state is predetermined.
  - $c(b)$ makes sense, $b(c)$ not so much (although you can invert the function).
  - "State variables" fully determine the trajectory of the system.
    - Exogenous state (income process) vs. endogenous state (capital) vs. interest rate

Dynamic Programming Step 1: Discretize the State Space

- Discretized grid space: `B_grid = LinRange(B_lower, B_upper, 100)`.
- First problem: the computer can only do finite things — note that we operate on an infinitely dimensional object.
  - Obs 1: In time, we avoid this problem by studying the recursive problem.
  - Obs 2: In grid space, remains tricky — `B_grid` $\neq [B_lower, B_upper]$.
  - Obs 3: $\log(0) = -\infty$ can cause problems.
    - Why not a problem for calculus? It will be for a computer...

Dynamic Programming Step 2: Iterate on the Functional Equation

- Write numerical code that does the following:

$$V_{n+1} = \max_{b'} \log(b - b') + \beta V_n(b') \tag{8}$$

- Algorithm:

  1. Start with any guess, for example $V_n = [0; 0; \ldots; 0]$. Is there a better guess?
  2. For every state, create "options" by computing flow utility + continuation value, and pick the highest $\rightarrow$ new guess V_{n+1}.
  3. Keep iterating and updating until $\max_b |V_n - V_{n+1}| < tol$.
  4. Once converged, you've solved for V and can produce the policy function $c(b)$.

Dynamic Programming Step 2: Iterate on the Functional Equation

- Example:
  1. Start with guess $V_0 = [0; 0; \ldots; 0]$.
  2. Suppose grid space in 0.1 steps: $[0.1, 0.2, \ldots, B_upper]$.
  3. Focus on one point in grid space, say $b = 0.5$.

     Given $b = 0.5$, try all consumption levels $c = 0.1, 0.2, 0.3, 0.4, 0.5$ with associated continuation values $\beta V_0(0.4), \beta V_0(0.3), \beta V_0(0.2), \beta V_0(0.1), \beta V_0(0)$.
  4. Pick the maximum — with our initial guess, it's likely $b' = 0.1$. Do you think in the second round of iteration the optimal pick would again be $b' = 0.1$?
  5. You do this state by state — two loops over b and b'.

Dynamic Programming Step 3 and 4

- Keep iterating until V_n and V_{n+1} are close everywhere!
- Get the policy function:
  - The value function is somewhat uninteresting (suppose I add a constant to flow utility).
  - What matters is $c(b)$ — that's where all the economics and data are.
  - Once you have V, compute $c(b)$: given b, find optimal $b' \to c(b) = b - b'(b)$.

Dynamic Programming Step 2: Issues and Sample Code

- Issues with discretization and bounds:
  - Obs 1: Infinite time — we avoid this problem by studying the recursive problem.
  - Obs 2: In grid space, note `B_grid` $\neq [B_lower, B_upper]$ — approximation.
  - Obs 3: $\log(0) = -\infty$ could pose problems — see what I do in the sample code.
    - Why not a problem for calculus, but computers struggle?

Dynamic Programming Step 2: Issues and Sample Code

```julia
## value function iteration
println("Starting value function iteration...")
iter = 0
diff = Inf
vals = zero(B_num) # this is just a temporary vector to store the values for each possible c
for round = 1:max_iter_HJB
    iter += 1
println("iter, " , iter)
    # Inside the main VFI loop, replace the inner scanning loop with:
    for i in 1:B_num
        B = B_grid[i]
        @inbounds for j in 1:i # can drop inbounds -- inessential but faster because julia s
            c = B - B_grid[j]
            # compute objective for candidate W' = Wgrid[j]
            vals[j] = u(c) + β * V[j]
        end
        #println("vals[: ",vals)
        # findmax over the valid slice 1:i
        best_val, best_idx = findmax(vals[1:i])
        Vnew[i] = best_val
        policy[i] = B - B_grid[best_idx]
    end

    diff = maximum(abs.(Vnew .- V))
    V, Vnew = Vnew, V   # swap references (cheap)
    if iter >= max_iter_HJB
        println("No convergence after ",iter," iteration.")
        convergence_V = false
    end

    if diff < tol_HJB
        println("Converged after ",iter," iteration.")
        convergence_V = true # boolean to check if converged
        break
    end
end

return (V=V,c_policy = policy,B_grid = B_grid,params=params,convergence_V=convergence_V)
```

Figure 3: Sample Code: Value Function Iteration

Dynamic Programming: Policy Function

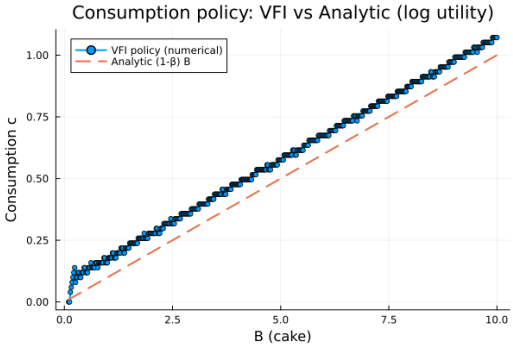

Figure 4: Sample Code: Policy Function $c(b)$

- I use $u(c = 0) = \log(0.1/3)$ — explore more in the problem set.
- More iteration doesn't solve the issue — it's not the same problem!

Preview Next Class

1. Neoclassical Model
2. Consumption–Saving Problem with Idiosyncratic Risk
3. (1) & (2) = Aiyagari Model